คู่มือตั้งค่าระบบสำหรับเจ้าของเว็บ

ภาษาไทย / ทำตามได้ทีละขั้น

เปิดเว็บให้ใช้งานจริง รับอีเมลแจ้งเตือน เชื่อมต่อ LINE OA

สำหรับคนใช้คอมพิวเตอร์คล่อง
แม้ไม่เคยเขียนเว็บหรือเรียน Computer Science

01	02	03
Vercel + Firebase	Gmail	LINE + LIFF
ที่อยู่เว็บและฐานข้อมูล	อีเมลผู้ดูแลและผู้ยืม	เข้าเว็บและรับแจ้งเตือน

> **คุณทำขั้นตอนในหน้าเว็บได้เป็นส่วนใหญ่ งานกำหนดสิทธิ์แอดมินและติดตั้งกฎฐานข้อมูลมีกรอบส่งต่อให้ผู้ดูแลเทคนิค พร้อมข้อความและคำสั่งให้ใช้โดยไม่ต้องเดา**

ใช้กับ repository: skullgamer2405-arch/smooa-equipment
LINE OA ที่ใช้: @474ezowf
อ่านและตั้งค่าในระบบทดสอบก่อนเปิดให้ผู้ใช้ทั่วไป

อ่านหน้านี้ก่อน แล้วค่อยลงมือ

คู่มือนี้เป็นขั้นตอนตั้งค่า ไม่ได้หมายความว่าบัญชีจริงเชื่อมสำเร็จแล้ว ชื่อเมนูอาจเปลี่ยนเล็กน้อยตามภาษาและรุ่นของผู้ให้บริการ ภาพผังเป็นภาพอธิบาย ไม่ใช่ภาพหน้าจอของบัญชีคุณ

เตรียมบัญชี สิทธิ์ และตรวจค่าใช้จ่าย 03

แยกระบบทดสอบกับระบบจริง 04

ตั้งค่า Vercel และกรอกค่าตัวแปร 05

ตั้งค่า Firebase และส่งต่องานสิทธิ์ 07

เชื่อม Gmail ด้วย App Password 11

สร้าง LINE Developers, LINE Login และ LIFF 13

ตรวจค่าครบ แล้วสร้างเว็บรุ่นใหม่ 17

ใส่ Rich Menu และทดสอบอีเมล 18

ทดสอบตั้งแต่จองจนถึงแจ้งเตือน 20

เปิดใช้งานจริงและย้อนกลับเมื่อมีปัญหา 22

วิธีแก้ปัญหาที่พบบ่อย 23

ภาคผนวกสำหรับผู้ดูแลเทคนิค 25

ใบบันทึกค่าและเอกสารอ้างอิง 26

> ลำดับที่แนะนำ: เตรียมบัญชี → เลือกโดเมน → Firebase → Gmail → LINE → กรอกค่าครบ → Redeploy → ทดสอบ → เปิดจริง อย่ารอให้ Deploy ผ่านตั้งแต่ตอนที่ค่ายังไม่ครบ

เตรียมอะไร และใครต้องมีสิทธิ์

สิ่งที่ต้องมี	ใช้ทำอะไร / สิทธิ์ที่ต้องมี
GitHub + Vercel	ดู repository นี้และจัดการโปรเจกต์ Vercel ได้ บัญชี Vercel ต้องได้รับสิทธิ์เข้าถึง repo
Google + Firebase	จัดการโปรเจกต์ Firebase, Authentication และสิทธิ์ Google Cloud ได้
Gmail ผู้ส่ง	เข้าบัญชีได้ เปิดการยืนยัน 2 ขั้นตอน และสร้าง App Password ได้
อีเมลผู้ดูแลผู้รับ	หนึ่งที่อยู่ที่เปิดอ่านได้ อาจเป็นบัญชีเดียวกับผู้ส่ง
LINE OA + LINE Developers	เป็นผู้ดูแล OA @474ezowf และ LINE Login channel ที่จะสร้าง
ผู้ดูแลเทคนิค	ช่วยตั้งสิทธิ์แอดมินและติดตั้งกฎ Firebase หน้าที่ 10 และ 25

> ตรวจค่าใช้จ่ายก่อน: Firebase ระบุว่าการใช้ Cloud Storage ต้องอยู่บนแผน Blaze และผูกบัญชีเรียกเก็บเงิน โดยเงื่อนไขมีผลตั้งแต่ 3 ก.พ. 2569 แม้ยังมีโควตาใช้ฟรีบางส่วน การใช้เกินโควตาอาจมีค่าใช้จ่าย หากไม่ยอมรับ อย่ากดอัปเกรดและให้ทีมทบทวนส่วนเก็บรูปก่อน

ตั้ง Budget alerts ใน Google Cloud เพื่อรับคำเตือนค่าใช้จ่าย การตั้งแจ้งเตือนไม่ใช่เพดานตัดการใช้งาน ตรวจโควตา Gmail, จำนวนข้อความ LINE OA และเงื่อนไข Vercel ให้ตรงลักษณะการใช้งานด้วย

> เก็บ App Password, ไฟล์ JSON และ access token ในตัวจัดการรหัสผ่านหรือ Vercel แบบ Secret ไม่ส่งในแชต ไม่ใส่ในคู่มือ และไม่อัปโหลดขึ้น GitHub

เข้าใจระบบทดสอบกับระบบจริง

คิดว่าเป็น “ร้านซ้อม” กับ “ร้านจริง” ค่าของแต่ละร้านต้องอยู่ด้วยกัน และต้องตรวจชื่อโปรเจกต์ก่อนบันทึกทุกครั้ง

พื้นที่	ใช้ทำอะไร
Preview ของโปรเจกต์หลัก	ดูหน้าตาและทดลองด้วยข้อมูลจำลอง ระบบตั้งใจไม่ส่งอีเมลหรือ LINE จริง ใส่ token แล้วก็ยังไม่ส่ง
โปรเจกต์ทดสอบแยก	ใช้ Vercel project และ Firebase project แยก ตั้ง Environment เป็น Production ของโปรเจกต์ทดสอบ เพื่อทดสอบบริการจริงกับบัญชีทดสอบ
Production ของโปรเจกต์หลัก	เว็บที่ผู้ใช้ทั่วไปจะใช้งาน เปิดหลังทดสอบผ่านเท่านั้น

ใช้ URL ให้ถูกชนิด

ตัวอย่างสมมติ	ใช้ที่ไหน
`https://smo-demo.vercel.app`	PUBLIC_APP_URL และ LIFF Endpoint แบบมี / ท้าย URL
`smo-demo.vercel.app`	Firebase Authorized domains: ชื่อโดเมนอย่างเดียว
`https://liff.line.me/1234567890-AbCdEf`	ปุ่ม Rich Menu: ใช้ LIFF ID จริงแทนตัวอย่าง
`https://vercel.com/...`	หน้าจัดการระบบ ไม่ใช่ URL สำหรับผู้ยืม

> แนะนำใช้ LIFF app และผู้รับทดสอบแยกด้วย อย่าแก้ LIFF Endpoint ของระบบจริงไปมาระหว่างทดลอง ค่าโดเมนทุกตัวอย่างในคู่มือต้องเปลี่ยนเป็นของคุณ

1. ตรวจโปรเจกต์ Vercel ให้ถูกตัว

vercel.com → เลือกทีม → โปรเจกต์ → Settings

1. ถ้ามีโปรเจกต์ `smooa-equipment` อยู่แล้ว ให้ใช้ตัวเดิมสำหรับระบบจริง สำหรับระบบทดสอบจึงค่อยสร้างโปรเจกต์ใหม่จาก repo เดียวกัน
2. ถ้าสร้างใหม่: เลือก `Add New → Project → Import Git Repository` แล้วเลือก `skullgamer2405-arch/smooa-equipment`
3. ตรวจ `Root Directory` เป็นราก repo คือ `.` ไม่ใช่ `smo-eq` และเลือก `Node.js 24.x` ในการตั้งค่า build
4. ให้ใช้ `vercel.json` ที่อยู่ใน repo ระบบมี 2 services: `app` คือหน้าเว็บ และ `api` คือส่วนทำงานเบื้องหลัง อย่าสร้าง service ชื่อ smo-eq เพิ่ม
5. เปิด `Settings → Domains` จดโดเมนถาวรของโปรเจกต์ ให้โดเมนทดสอบกับโดเมนจริงเป็นคนละชื่อ



> โค้ดชุดตั้งค่าล่าสุดอยู่ใน branch preview/ui-vercel-services และมี PR #1 ก่อนนำขึ้นจริงให้ตรวจว่า main รวมชุดนี้แล้ว โปรเจกต์ทดสอบอาจใช้ branch นี้เป็น Production Branch ของตัวเองได้ อย่าเปลี่ยน branch ของโปรเจกต์หลักเพื่อทดลองโดยไม่ตรวจผล

2. วิธีเพิ่มค่าตั้งค่าใน Vercel

Environment Variables คือช่องเก็บ “ชื่อค่า” และ “ค่าจริง” เว็บจะอ่านไปใช้ ไม่ใช่ช่องให้เขียนโปรแกรม

Vercel → โปรเจกต์ → Settings → Environment Variables

ตัวอย่างการกรอกหนึ่งรายการ

Key / Name	SMTP_HOST
Value	smtp.gmail.com
Environment	Production ของโปรเจกต์ที่กำลังตั้งค่า
Type	Config (รหัสลับให้เลือก Secret)

Save

1. กด Add / Add Environment Variable กรอกชื่อด้านซ้ายจากตารางให้ตรงทุกตัว ชื่อพิมพ์ใหญ่และเครื่องหมายขีดล่างต้องครบ
2. ช่อง Value ใส่ค่าจริงอย่างเดียว ไม่ใส่เครื่องหมายคำพูด ไม่ใส่ชื่อค่าซ้ำ และไม่ใส่คำว่า “ตัวอย่าง”
3. เลือก Production ของโปรเจกต์ที่ตั้งค่า ถ้ามีช่อง Service ให้ค่าขึ้นต้น `VITE_` ใช้กับ app และให้ api เห็นค่า Firebase ชุดเดียวกัน รหัสลับให้ api ใช้
4. ถ้าไม่มีช่องเลือก Service ใช้ตัวแปรระดับโปรเจกต์ได้ เลือกชนิด Secret สำหรับรหัสลับ บางหน้าจอรุ่นก่อนใช้คำว่า Sensitive แล้วกด Save

> ค่าที่ขึ้นต้น VITE_ จะถูกส่งไปกับหน้าเว็บ ห้ามนำรหัสลับไปตั้งชื่อ VITE_ การบันทึกค่าไม่เปลี่ยนเว็บรุ่นที่กำลังเปิด ต้อง Redeploy เมื่อกรอกครบ ดูหน้า 17

3. หา Firebase config มาใส่ให้ครบ

console.firebase.google.com → เลือก project → รูปเฟือง → Project settings

1. ใช้ Firebase project ที่ตั้งใจเลือก ถ้าสร้างใหม่ให้ตั้งชื่อแยกระบบทดสอบ/ระบบจริง และจด `Project ID` ซึ่งเป็นรหัสอ้างอิง ไม่ใช่แค่ชื่อที่แสดง
2. แท็บ `General → Your apps` เลือกแอปเว็บ ถ้ายังไม่มีให้กดรูป </> ตั้งชื่อแล้ว Register app งานนี้ไม่ต้องเปิด Firebase Hosting เพราะเว็บอยู่ Vercel
3. เลือก `SDK setup and configuration → Config` คุณจะเห็นชุดค่า เช่น apiKey และ projectId ให้คัดลอกเฉพาะค่าด้านในเครื่องหมายคำพูด

ชื่อที่ใส่ใน Vercel	คัดลอกจาก Firebase
`VITE_FIREBASE_API_KEY`	`apiKey`
`VITE_FIREBASE_AUTH_DOMAIN`	`authDomain`
`VITE_FIREBASE_PROJECT_ID`	`projectId`
`VITE_FIREBASE_STORAGE_BUCKET`	`storageBucket`
`VITE_FIREBASE_APP_ID`	`appId`
`FIREBASE_STORAGE_BUCKET`	ค่าเดียวกับ storageBucket อีกหนึ่งรายการสำหรับ API

> ค่าชุดหน้าเว็บนี้เปิดเผยได้ตามการทำงานของ Firebase แต่ไม่ใช่สิทธิ์ผู้ดูแล ความปลอดภัยอยู่ที่กฎและการตรวจบัญชี ส่วนไฟล์ service account JSON ในหน้า 9 เป็นคนละอย่างและเป็นความลับ

ชื่อ bucket อาจลงท้าย .appspot.com หรือ .firebasestorage.app ให้ใช้ตาม config จริง ห้ามเดาหรือเปลี่ยนท้ายชื่อเอง

4. เปิดบริการ Firebase ที่เว็บใช้

1. `Build → Authentication → Get started → Sign-in method`: เปิด Email/Password แล้ว Save ไม่ต้องเปิด Phone หรือ Google เพื่อให้เว็บชุดนี้ทำงาน
2. `Authentication → Settings → Authorized domains`: เพิ่มโดเมนเว็บทดสอบ/เว็บจริงที่ใช้ เช่น `smo-demo.vercel.app` ไม่ใส่ https:// หรือ / ต่อท้าย
3. `Build → Firestore Database`: ถ้ายังไม่มีให้ Create database แบบ Production mode สำหรับฐานข้อมูลเริ่มต้น `(default)` เลือกภูมิภาคให้เหมาะกับผู้ใช้และทีมตั้งแต่ต้น หากมีแล้วให้ใช้ของเดิม ไม่สร้างทับ
4. `Build → Storage`: ตรวจว่า bucket พร้อม หากขออัปเกรดเป็น Blaze ให้ตรวจค่าใช้จ่ายกับเจ้าของก่อนดำเนินการ ดูข้อจำกัดหน้า 3
5. หากสร้าง Storage หลังลงทะเบียนแอปแล้ว ให้กลับไปหน้า config อีกครั้ง ตรวจชื่อ bucket แล้วแก้ค่าทั้ง `VITE_FIREBASE_STORAGE_BUCKET` และ `FIREBASE_STORAGE_BUCKET` ให้ตรงกัน

> อย่าเลือก Test mode หรือเขียนกฎ allow read, write: if true เพื่อแก้ปัญหาชั่วคราว ต้องให้ผู้ดูแลเทคนิคติดตั้ง rules และ indexes จาก repo ก่อนเปิดให้คนใช้จริง

เมื่อทำจบ ควรได้อะไร

- [ ] Email/Password แสดง Enabled
- [ ] ชื่อโดเมนถูกเพิ่มใน Authorized domains
- [ ] Firestore และ Storage พร้อมใช้งานใน project เดียวกัน
- [ ] จด Project ID และชื่อ bucket ไว้ โดยยังไม่มีข้อมูลผู้ยืมจริงในระบบทดสอบ

5. ให้ API ติดต่อ Firebase ได้

`Service account` เปรียบเหมือนบัญชีพนักงานของเว็บ ใช้อ่านและบันทึกข้อมูล ส่วนไฟล์กุญแจ JSON คือรหัสลับของบัญชีนี้ ไม่ใช่บัญชีแอดมินที่คุณใช้ล็อกอินหน้าเว็บ

> console.cloud.google.com → เลือก project เดียวกับ Firebase → IAM & Admin → Service Accounts

1. ให้ผู้มีสิทธิ์ Google Cloud สร้าง service account สำหรับ API ของเว็บ เช่น `smo-web-api`
2. กำหนดสิทธิ์เท่าที่จำเป็น: `Cloud Datastore User`, `Firebase Authentication Viewer` และ `Storage Object User` เฉพาะ bucket ที่ใช้งาน ให้ผู้ดูแลเทคนิคช่วยตรวจขอบเขต ไม่ใช้ Owner เป็นทางลัด
3. เปิดบัญชีที่สร้าง → `Keys → Add key → Create new key → JSON` ดาวน์โหลดไฟล์ไว้ในที่ปลอดภัย หากองค์กรห้ามสร้าง key ให้ผู้ดูแลจัดวิธีที่องค์กรอนุมัติ ไม่เปลี่ยนนโยบายหลบข้อจำกัด
4. เปิดไฟล์ด้วยโปรแกรมข้อความ เช่น Notepad คัดลอกเนื้อหาทั้งหมดรวมวงเล็บปีกกา ตั้งใน Vercel ชื่อ `FIREBASE_SERVICE_ACCOUNT_JSON` เลือกชนิด Secret
5. วาง JSON ต้นฉบับครบชุด ไม่เพิ่มเครื่องหมายคำพูดครอบทั้งไฟล์ และไม่แก้ส่วน private_key เก็บสำเนาในที่ปลอดภัยแล้วนำออกจากโฟลเดอร์แชร์/Downloads ที่ผู้อื่นเข้าถึงได้

> Project ID ใน service account ต้องตรงกับ VITE_FIREBASE_PROJECT_ID กุญแจนี้ห้ามใส่ GitHub หรือ VITE_ และไม่ใช้รหัสนี้แทนรหัสผ่านหน้าแอดมิน

หากทำไฟล์หลุด ให้เพิกถอน key นั้นใน Google Cloud สร้างใหม่ เปลี่ยน Secret ใน Vercel และ Redeploy การลบข้อความหรือไฟล์ที่เผยแพร่อย่างเดียวไม่พอ

6. สร้างแอดมินและติดตั้งกฎข้อมูล

มีบัญชี Firebase แล้ว ยังไม่ได้หมายความว่าเป็นแอดมิน ต้อง “ยืนยันอีเมล” และมีสิทธิ์ `admin=true` ที่ออกจากระบบฝั่งเซิร์ฟเวอร์ด้วย

1. ใน `Firebase → Authentication → Users → Add user` สร้างบัญชีอีเมลของผู้ดูแล ตั้งรหัสผ่านเฉพาะระบบ เก็บรหัสอย่างปลอดภัย ไม่ใช้รหัส Gmail/App Password แทน
2. เปิดรายละเอียดบัญชี คัดลอก `UID` และตรวจอีเมลให้ตรงคน ส่ง UID กับ Project ID ให้ผู้ดูแลเทคนิคผ่านช่องทางภายใน ไม่ต้องส่งรหัสผ่าน
3. ให้ผู้ดูแลเทคนิคสร้างลิงก์ยืนยันอีเมลด้วย Firebase Admin SDK แล้วส่งให้เจ้าของบัญชีเป็นการส่วนตัว เจ้าของเปิดลิงก์ยืนยัน ห้ามใช้การรีเซ็ตรหัสผ่านแทนการยืนยันอีเมล
4. ให้ผู้ดูแลเทคนิคใช้สคริปต์ใน repo ตั้ง `admin=true` ให้ UID ที่ตรวจแล้ว จากนั้นคุณออกจากเว็บและเข้าสู่ระบบใหม่
5. ก่อนทดสอบจอง ให้ผู้ดูแลเทคนิคติดตั้ง Firestore rules, indexes และ Storage rules รวมถึงตรวจข้อมูลปฏิทินเดิมและ TTL ตามหน้า 25

ข้อความส่งให้ผู้ดูแลเทคนิค

“ขอให้ช่วยยืนยัน project และ UID ของบัญชีผู้ดูแล สร้างลิงก์ยืนยันอีเมล ตั้ง admin claim และ deploy Firebase rules/indexes/Storage ของ repo นี้ กรุณาทำ dry run/สำรองก่อนแก้ข้อมูล ดูคำสั่งในหน้า 25”

> ถ้าไม่มีผู้ดูแลเทคนิค ให้หยุดเปิดใช้จริงที่ขั้นตอนนี้ก่อน อย่าแก้หน้าจอให้ข้ามการตรวจสิทธิ์ และอย่าเปิดฐานข้อมูลเป็นสาธารณะเพื่อให้ล็อกอินได้

7. สร้าง App Password ของ Gmail

`Gmail ผู้ส่ง` คือบัญชีที่เว็บใช้ส่งเมล `อีเมลผู้รับ` คือคนรับคำขอใหม่ ทั้งสองเป็นบัญชีเดียวกันได้ และไม่ใช่การให้สิทธิ์แอดมินหน้าเว็บ

1. เปิด myaccount.google.com/security ตรวจรูปโปรไฟล์มุมบนให้แน่ใจว่าอยู่ในบัญชี Gmail ที่จะใช้ส่ง แนะนำบัญชีของทีมที่เจ้าของอนุญาต
2. เปิด `2-Step Verification / การยืนยันแบบ 2 ขั้นตอน` และทำตามคำแนะนำของ Google จนเปิดใช้งานสำเร็จ
3. เปิด myaccount.google.com/apppasswords อาจต้องลงชื่อเข้าใช้อีกครั้ง ตรวจว่าบัญชียังเป็นผู้ส่งตัวเดิม
4. ตั้งชื่อแอปให้จำได้ เช่น `SMO OA email` แล้วกด Create Google จะแสดงรหัส 16 ตัว
5. คัดลอกรหัสไปใส่ `SMTP_PASS` ใน Vercel แบบ Secret โดยเอาช่องว่างที่ Google ใช้แบ่งกลุ่มออก ใช้รหัสนี้แทนรหัสผ่าน Google ปกติ

> หากไม่มี App passwords: บัญชีอาจถูกนโยบายองค์กรจำกัด เปิด Advanced Protection หรือใช้รูปแบบยืนยันตัวตนที่ไม่รองรับ ให้ผู้ดูแล Google Workspace ตรวจ อย่าปิดการยืนยัน 2 ขั้นตอนหรือเปิด Less secure apps เพื่อแก้ปัญหา

ข้อควรรู้

เมื่อเปลี่ยนรหัสผ่าน Google รหัสสำหรับแอปอาจถูกเพิกถอน หากเมลหยุดส่งหลังเปลี่ยนรหัส ให้สร้าง App Password ใหม่ เปลี่ยน Secret แล้ว Redeploy ส่วนการส่งเมลยังอยู่ภายใต้โควตาของ Gmail/Workspace

8. ใส่ค่า Gmail ใน Vercel

Vercel → โปรเจกต์ที่กำลังตั้งค่า → Settings → Environment Variables

Key / Name	Value ที่ใส่
`ADMIN_NOTIFICATION_EMAIL`	อีเมลผู้ดูแลผู้รับหนึ่งที่อยู่ เช่น `owner@example.com` เปลี่ยนเป็นของจริง
`SMTP_HOST`	`smtp.gmail.com`
`SMTP_PORT`	`587`
`SMTP_SECURE`	`false` พอร์ตนี้ใช้ STARTTLS เข้ารหัส ไม่ได้ส่งรหัสผ่านแบบข้อความเปล่า
`SMTP_USER`	Gmail ของผู้ส่งแบบเต็ม
`SMTP_PASS`	App Password 16 ตัว ไม่มีช่องว่าง เลือก Secret
`PUBLIC_APP_URL`	URL เว็บของโปรเจกต์นี้ เช่น `https://smo-demo.vercel.app`

> SMTP_FROM เป็นตัวเลือก ไม่ใส่ก็ได้ ระบบใช้ SMTP_USER เป็นผู้ส่ง ถ้าระบุเองต้องเป็นที่อยู่ที่ผู้ให้บริการอนุญาต

ตั้งเสร็จแล้วจะเกิดอะไร

เหตุการณ์	ส่งไปหาใคร
ผู้ยืมส่งคำขอใหม่	อีเมล ADMIN_NOTIFICATION_EMAIL
ผู้ดูแลอนุมัติ / ปฏิเสธ	อีเมลของผู้ยืมที่อยู่ในคำขอนั้น
เตือนคืนอุปกรณ์	ใช้ LINE อีกระบบหนึ่ง ไม่ใช่การเตือนทาง Gmail ในชุดนี้

ยังไม่ต้องกดทดสอบตอนนี้ ให้ทำ LINE และกรอกค่าที่เหลือจนถึงหน้า 17 ก่อน จากนั้นตรวจอีเมลในหน้า 19

9. เริ่ม LINE ด้วย Provider เดียวกัน

LINE มีชื่อหลายแบบ แต่จำเพียงบทบาทนี้ เพื่อไม่คัดลอก ID หรือ token สลับช่อง

ชื่อใน LINE	นึกภาพง่าย ๆ
OA @474ezowf	แชทของสโมสรที่ผู้ใช้เพิ่มเพื่อนและรับข้อความ
Provider	แฟ้มขององค์กร เก็บทั้ง Messaging API และ LINE Login ไว้ด้วยกัน
Messaging API channel	ช่องทางให้เว็บส่งข้อความผ่าน OA มี access token ที่เป็นความลับ
LINE Login channel	ช่องทางขอให้ผู้ใช้เชื่อมบัญชีและอนุญาตสิทธิ์ มี Channel ID
LIFF app	ตัวเปิดเว็บใน LINE มี LIFF ID และ LIFF URL สำหรับ Rich Menu

1. เข้า manager.line.biz เลือก OA `@474ezowf` → `Settings` → `Messaging API` → `Enable`
2. เลือก Provider ขององค์กร จดชื่อให้ชัด หากมี Messaging API อยู่แล้วให้ใช้ของเดิม และตรวจ Provider อย่าสร้างซ้ำ
3. เข้า LINE Developers Console เลือก Provider นั้น เปิด Messaging API channel ของ OA → แท็บ `Messaging API`
4. ที่ `Channel access token (long-lived)` เลือก Issue เมื่อยังไม่มี แล้วเก็บใน Vercel ชื่อ `LINE_MESSAGING_CHANNEL_ACCESS_TOKEN` แบบ Secret

> LINE Login ที่สร้างหน้าถัดไปต้องอยู่ใน Provider เดียวกับ OA จึงเชื่อมกันได้ งานนี้ไม่ใช้ LINE Notify และไม่ต้องเปิด Webhook รับแชทเพื่อให้ส่งแจ้งเตือนได้

10. สร้าง LINE Login แล้วผูกกับ OA

LINE Developers Console → Provider เดียวกับ OA → Create a new channel → LINE Login

1. เลือกประเภทแอป `Web app` กรอกชื่อและคำอธิบายที่ผู้ใช้รู้ว่าเป็นเว็บยืมอุปกรณ์ของสโมสร ใส่อีเมลติดต่อที่ทีมอ่านจริง และทำตามฟิลด์ที่ LINE กำหนด
2. เมื่อตั้ง channel สำเร็จ เปิดแท็บ `Basic settings` คัดลอก `Channel ID` ของ LINE Login ไว้ใช้เป็น `LINE_LOGIN_CHANNEL_ID`
3. ใน `Basic settings → Linked LINE Official Account → Edit` เลือก OA `@474ezowf` แล้ว Update
4. ถ้าเลือก OA ไม่ได้ ให้ตรวจว่าอยู่ Provider เดียวกัน และบัญชีคุณเป็นผู้ดูแลทั้ง OA และ LINE Login channel ไม่แก้ด้วยการสร้าง OA ใหม่
5. ช่วงทดสอบที่สถานะ `Developing` ให้เพิ่มบัญชีผู้ทดสอบใน `Roles` และให้ผู้รับยอมรับคำเชิญ บัญชีนอกสิทธิ์อาจยังเข้าไม่ได้
6. หลังทดสอบครบแล้ว ค่อยเปลี่ยน LINE Login channel เป็น `Published` ก่อนเปิดให้ผู้ใช้ทั่วไป การเผยแพร่ channel ไม่เท่ากับเปิด Rich Menu อัตโนมัติ

> อย่าสลับ Channel ID ของ LINE Login กับของ Messaging API และไม่ต้องคัดลอก Channel secret มาใส่เว็บชุดนี้ ไม่ต้องเดา Callback URL เพิ่ม: ขั้นตอนนี้ใช้ LIFF ที่จะสร้างหน้า 15

หาก LINE ขอ URL นโยบายความเป็นส่วนตัว/เงื่อนไข ให้ใช้เอกสารจริงที่อธิบายการใช้ข้อมูลขององค์กร ห้ามใส่ URL สมมติเพื่อให้ผ่านหน้าสมัคร

11. สร้าง LIFF สำหรับเปิดเว็บใน LINE

LINE Login channel → แท็บ LIFF → Add

ช่องที่เจอ	ค่าที่ควรเลือก
LIFF app name	SMO OA Equipment หรือชื่อที่ทีมจำได้
Size	`Full`
Endpoint URL	โดเมนถาวรของโปรเจกต์นี้ เช่น `https://smo-demo.vercel.app/` ใส่ / ท้าย URL
Scope	เลือก `openid` และ `profile` ไม่ต้องขอ email หรือ chat_message.write สำหรับงานนี้
Add friend option	เลือกเปิด เช่น `On (normal)` เพื่อเสนอเพิ่มเพื่อน OA ที่เชื่อมไว้

1. กด Add / Save แล้วคัดลอก LIFF ID เช่น `1234567890-AbCdEf` เก็บเป็น `VITE_LINE_LIFF_ID` ค่า ID นี้ไม่ใช่ token ลับ
2. คัดลอก LIFF URL เช่น `https://liff.line.me/1234567890-AbCdEf` ไว้ทำปุ่ม Rich Menu
3. Endpoint ต้องเปิดเว็บได้โดยไม่มีหน้าให้ล็อกอิน Vercel ขวาง ให้ตรวจ Settings → Deployment Protection กับผู้ดูแล แก้เฉพาะเว็บทดสอบที่ตั้งใจให้มือถือเข้าถึง และอย่าใส่ bypass token ใน URL เมนู

> Endpoint URL คือปลายทางเว็บบน Vercel ส่วน LIFF URL คือลิงก์ที่ LINE ใช้เปิดเว็บ ปุ่ม Rich Menu ควรใช้ LIFF URL ไม่ใช่ URL หน้า dashboard หรือ Preview ชั่วคราว

เปลี่ยนโดเมนหรือ LIFF ID ภายหลัง ต้องอัปเดตค่าที่เกี่ยวข้องและ Redeploy อย่าให้ LIFF ทดสอบชี้เว็บจริงโดยไม่ตั้งใจ

12. ใส่ค่า LINE และตั้งเวลางาน

Vercel → โปรเจกต์ที่กำลังตั้งค่า → Settings → Environment Variables

Key / Name	Value ที่ใส่
`VITE_LINE_LIFF_ID`	LIFF ID จากหน้าที่ 15 เช่น `1234567890-AbCdEf`
`VITE_LINE_OA_URL`	`https://line.me/R/ti/p/@474ezowf`
`LINE_LOGIN_CHANNEL_ID`	Channel ID ของ LINE Login เป็นตัวเลข
`LINE_MESSAGING_CHANNEL_ACCESS_TOKEN`	Channel access token ของ Messaging API เลือก Secret
`LINE_ENABLED`	`true` เมื่อกำลังเตรียมทดสอบการเชื่อมต่อครบชุด
`CRON_SECRET`	รหัสสุ่มอย่างน้อย 32 ตัวจากตัวจัดการรหัสผ่าน เลือก Secret ไม่ใช้รหัสล็อกอินหรือ token อื่นแทน

> CRON_SECRET คือรหัสให้ Vercel ยืนยันว่าเป็นผู้เรียกงานเตือนคืน ระบบส่งผ่าน Authorization header ให้เอง คุณไม่ต้องใส่รหัสใน URL หรือส่งรหัสให้ผู้ยืม

งานเตือนคืนตั้งไว้แล้วในโค้ด

ตั้งวันละหนึ่งครั้งที่ `0 2 * * *` ซึ่งใช้เวลา UTC เท่ากับช่วง 09:00 น. เวลาไทย บน Vercel Hobby งานอาจเริ่มระหว่าง 09:00-09:59 น. ไม่รับประกันตรงนาที ทำงานเฉพาะ Production ของโปรเจกต์นั้น

เลือก environment ให้ถูก หากเป็นโปรเจกต์ทดสอบต้องใช้ค่า LINE/LIFF ของการทดสอบ ตรวจว่า Cron ของระบบทดสอบจะไม่ส่งหาผู้ใช้จริง

13. ตรวจค่าครบ แล้ว Redeploy

โค้ดจะหยุด Production build ถ้าค่าที่จำเป็นยังขาด นี่เป็นตัวช่วยป้องกันการเปิดเว็บไม่พร้อม ไม่ควรปิดตัวตรวจเพื่อให้ผ่าน

- ☐ Firebase: ค่า VITE_FIREBASE_* ครบ 5 รายการ และชื่อ bucket ฝั่ง API ตรงกัน
- ☐ Firebase: FIREBASE_SERVICE_ACCOUNT_JSON เป็น Secret ของ project เดียวกัน
- ☐ Gmail: ADMIN_NOTIFICATION_EMAIL และ SMTP ทั้ง 5 รายการครบ
- ☐ LINE: LIFF ID, Login Channel ID และ Messaging access token ไม่สลับกัน
- ☐ LINE_ENABLED=true, CRON_SECRET ยาวอย่างน้อย 32 ตัว และ PUBLIC_APP_URL เป็น HTTPS ของเว็บโปรเจกต์นี้
- ☐ ไม่มี SMO_DEMO_MODE=true หรือ VITE_DEMO_MODE=true ใน Production และไม่มีรหัสลับในชื่อที่ขึ้นต้น VITE_

1. ใน Vercel เปิด `Deployments` เลือกรุ่นที่ต้องการของโปรเจกต์นี้ ตรวจ branch/commit และ Environment ก่อน
2. เลือกเมนู `Redeploy` แล้วรอจนขึ้น `Ready` หากรุ่นนั้นเป็น Preview การ Redeploy จะยังเป็น Preview ไม่กลายเป็น Production
3. ระบบทดสอบต้องสร้าง deployment จาก Production Branch ของโปรเจกต์ทดสอบ ระบบจริงใช้ main หลังรวมโค้ดที่ผ่านการตรวจแล้ว ดูขั้นตอนเปิดจริงหน้า 22
4. เปิดโดเมนจาก Settings → Domains แล้วลอง refresh หน้าย่อย เช่น `/equipment.html` และ `/admin.html` ให้เข้าได้

> ถ้าขึ้น MISSING/INVALID ให้ดูชื่อตัวแปรใน Build Logs แก้ชื่อนั้นแล้ว Redeploy ใหม่ การขึ้น Ready หมายถึงสร้างเว็บผ่าน ไม่ได้ยืนยันว่า Gmail หรือ LINE ส่งถึงจริงแล้ว

14. สร้าง Rich Menu แบบ 3 ช่อง

เข้า LINE Official Account Manager เลือก OA ที่ถูกต้อง → `Rich menus / ริชเมนู` (อาจอยู่ใต้ Chat) → Create เลือกแบบสั้น 3 ช่องเท่ากัน



ภาพเมนูที่เตรียมไว้ในโปรเจกต์ ขนาด 2500 × 843 พิกเซล

ดาวน์โหลดภาพจาก ลิงก์ไฟล์ภาพ Rich Menu เปิดแล้ว Save image as เก็บในเครื่อง จากนั้นอัปโหลดเป็นภาพเมนู

ช่องในภาพ	Action = Link / URL
ยืมอุปกรณ์	`https://liff.line.me/YOUR_LIFF_ID/equipment.html`
ปฏิทิน	`https://liff.line.me/YOUR_LIFF_ID/calendar.html`
วิธีใช้งาน	`https://liff.line.me/YOUR_LIFF_ID/`

1. เปลี่ยน `YOUR_LIFF_ID` ทั้ง 3 ลิงก์เป็น LIFF ID จริง ไม่ต้องใส่เครื่องหมาย < >
2. ตั้งข้อความแถบเปิดเมนูเป็น `ยืมอุปกรณ์` ตั้งช่วงเวลาแสดงผล และเลือกให้เมนูแสดงตามที่ต้องการ
3. ระหว่างยังทดสอบ ให้ใช้ OA ทดสอบ หรือเปิด LIFF URL ให้ผู้ทดสอบโดยตรง ไม่ต้องเปิดเมนูใหม่ทับให้ผู้ใช้จริงทั้งหมด เมื่อผ่านจึงบันทึก/เปิดแสดง Rich Menu ของจริง

> ช่อง “วิธีใช้งาน” ในชุดนี้พาไปหน้าแรกของเว็บ ไม่ได้เปิด PDF อัตโนมัติ ถ้าต้องการปุ่มเปิดคู่มือ PDF ต้องมี URL เผยแพร่ของคู่มือนั้นแล้วค่อยเปลี่ยนปลายทาง

15. ทดสอบอีเมลจากหน้าแอดมิน

เว็บทดสอบ → /admin.html → เข้าสู่ระบบ → การตั้งค่า → อีเมลแจ้งเตือน

1. ใช้บัญชีผู้ดูแล Firebase ที่ยืนยันอีเมลและตั้งสิทธิ์แล้ว ห้ามใช้บัญชีทดลองจาก Preview เพื่อทดสอบ Gmail จริง
2. ตรวจบรรทัด `ผู้รับคำขอใหม่` ต้องเป็นอีเมลผู้ดูแลที่ตั้งไว้ และ `ช่องทางส่ง` ต้องเป็น SMTP
3. ช่อง `URL ของ Google Apps Script` ให้เว้นว่าง แล้วกด `บันทึกการตั้งค่า` ถ้ามี URL อยู่ ระบบจะเลือก Apps Script แทน Gmail SMTP
4. เมื่อขึ้น `ตั้งค่าครบแล้ว - ส่งทดสอบเพื่อยืนยันการเชื่อมต่อ` ให้กด `ส่งอีเมลทดสอบ` หนึ่งครั้ง
5. เปิดกล่องอีเมลของผู้รับ ตรวจ Inbox, Spam และ All Mail ต้องเห็นอีเมลทดสอบจริง ไม่ใช้ข้อความ “บริการตอบรับแล้ว” เป็นหลักฐานแทนการเปิดดูเมล
6. ลองส่งคำขอยืมใหม่ด้วยข้อมูลทดสอบ ตรวจว่าผู้ดูแลได้เมล มีวันยืมคืนถูกต้อง และลิงก์แอดมินเปิดเว็บของโปรเจกต์นี้

> อีเมลที่เกิดก่อนตั้งค่าไม่ถูกส่งย้อนหลังอัตโนมัติ หากผลการส่งไม่ชัดเจน ให้ตรวจคิวและกล่องเมลก่อนกดใหม่ เพื่อไม่ให้ส่งซ้ำ

สิ่งที่ควรตรวจเพิ่ม

- [ ] อนุมัติหรือปฏิเสธแล้ว ผู้ยืมทดสอบได้รับอีเมลผลพิจารณา
- [ ] อีเมลผู้รับคำขอใหม่กับผู้รับผลพิจารณาไม่สลับกัน
- [ ] กดลิงก์ในอีเมลแล้ว ยังต้องล็อกอินแอดมินตามปกติ ไม่อนุมัติทันที

16. ทดสอบ LINE ให้ครบหนึ่งรอบ

ใช้มือถือจริงและบัญชีทดสอบ หากทำได้ให้ทดสอบทั้ง iPhone และ Android และเลือกอุปกรณ์ทดสอบที่ไม่ชนกับการใช้งานจริง

1. เพิ่มเพื่อน OA และตรวจว่าไม่ได้บล็อก เปิดแชท → Rich Menu → ยืมอุปกรณ์ หากยังไม่เปิดเมนูให้ใช้ LIFF URL สำหรับผู้ทดสอบ
2. ครั้งแรกอ่าน popup ของเว็บแล้วกด `ยินยอมและเชื่อมต่อ LINE` ทำตามหน้าขออนุญาตของ LINE และเพิ่มเพื่อนเมื่อระบบขอ
3. ปิดเว็บแล้วเปิดจากเมนูใหม่ด้วยมือถือและเบราว์เซอร์เดิม เมื่อเชื่อมสำเร็จแล้ว popup ของเว็บไม่ควรถามซ้ำ หน้าอื่นไม่มีแผงให้เชื่อม LINE เพิ่ม
4. เลือกอุปกรณ์ กรอกข้อมูลทดสอบและวันยืมคืน ส่งคำขอ แล้วจดรหัสคำขอไว้ สถานะตอนนี้ต้องเป็นรออนุมัติ ไม่ใช่ได้รับอนุมัติทันที
5. ผู้ดูแลเปิดแอดมิน → อนุมัติคำขอทดสอบ ยืนยันว่าอีเมลผู้ดูแลได้รับคำขอ และบัญชี LINE ผู้ยืมคนนั้นได้รับข้อความอนุมัติ
6. เปิด `รายการของฉัน` ด้วย LINE บัญชีเดิม ตรวจว่าพบคำขอของตัวเอง จากนั้นให้ผู้ดูแลบันทึกรับคืนเมื่อทดสอบเสร็จ

> เลือก “ไว้ภายหลัง” ได้ แต่คำขอที่ไม่ผูก LINE จะไม่มีแจ้งเตือน LINE และไม่ถูกจับคู่ย้อนหลังจากชื่อหรืออีเมล

หากเปลี่ยนเครื่อง/โดเมน ล้างข้อมูลเว็บ ถอนสิทธิ์ หรือ LINE ขอสิทธิ์เพิ่ม อาจต้องอนุญาตใหม่ การปฏิเสธคำขอแจ้งผ่านอีเมลตามระบบปัจจุบัน ยังไม่มีข้อความปฏิเสธทาง LINE

17. ตรวจการเตือนคืนและงานประจำ

ระบบส่งเตือนเฉพาะรายการที่อนุมัติแล้ว ยังไม่คืน และผูก LINE ไว้ ส่งก่อนวันครบกำหนด 1 วัน กับวันครบกำหนด ช่วง `09:00-09:59 น. เวลาไทย` บน Vercel Hobby

ตัวอย่างกำหนดคืน	คาดหวังผล
คืน 15 พฤศจิกายน	มีเตือนวันที่ 14 และ 15 ช่วงเช้าตามรอบงาน
บันทึกรับคืนก่อนรอบเตือน	ไม่ควรมีคำเตือนของรายการนั้นหลังรับคืน
ไม่ได้ผูก LINE / บล็อก OA	อาจไม่ได้รับข้อความ ต้องตรวจสถานะในเว็บด้วย
เกินกำหนดคืนไปแล้ว	ชุดนี้ไม่ได้ส่งเตือนค้างคืนซ้ำทุกวัน

1. ใน Vercel โปรเจกต์ทดสอบ เปิด `Settings → Cron Jobs` ตรวจว่ามี path `/api/line/reminders` และเวลา `0 2 * * *` ถ้าเมนูย้ายให้ค้น Cron Jobs
2. สร้างรายการทดสอบที่ครบกำหนด “วันพรุ่งนี้” และอีกหนึ่งรายการที่ “วันนั้น” ให้พร้อมก่อนรอบ 09:00 ใช้ผู้รับทดสอบเท่านั้น แล้วรอผลจนพ้น 09:59
3. เปิดหน้า `Logs` ของ Vercel ให้ผู้ดูแลตรวจผลรอบ Cron และงานที่ค้าง หากต้องรันซ้ำให้ผู้ดูแลเทคนิคทำอย่างปลอดภัย ไม่เปิด URL พร้อมรหัสลับให้ผู้อื่น
4. ทดสอบบันทึกรับคืนก่อนรอบเตือนด้วยหนึ่งรายการ ต้องไม่ส่งซ้ำหลังคืน และตรวจโควตาข้อความใน OA Manager เป็นระยะ

> “sent” หมายถึงบริการ LINE รับคำขอ ไม่ยืนยันว่าผู้ใช้เห็นหรืออ่านแล้ว งานล้มเหลวอาจไม่ถูก retry ทันที อย่ารอพรุ่งนี้อย่างเดียว ให้ตรวจคิว needs_review กับผู้ดูแล

Cron รันเฉพาะ Production ของโปรเจกต์นั้น การเปิดหน้า Preview ค้างไว้หรือปิดคอมพิวเตอร์ไม่ใช่สิ่งควบคุมการทำงานของ Cron

18. เปิดใช้งานจริงเมื่อทดสอบผ่าน

ทำขั้นตอนนี้กับโปรเจกต์หลักหลังทดสอบครบ ใช้รายการเดิมเป็นตัวตรวจ อย่าคัดลอกรหัสหรือ Project ID ของระบบทดสอบมาปะปน

1. บันทึกเว็บรุ่นเดิมที่ใช้ได้และสำรองข้อมูล/กฎ Firebase ก่อนเปลี่ยน ให้ผู้ดูแลเทคนิคยืนยันแผนย้อนกลับ
2. กรอกค่าจริงใน Environment `Production` ของโปรเจกต์หลัก ตรวจโดเมนจริง, Firebase จริง, Gmail จริง และ LIFF จริงให้ตรงกัน รวมถึง Authorized domains และ LIFF Endpoint
3. ให้ผู้ดูแลเทคนิคตรวจว่าโค้ด PR #1 ผ่าน CI และถูกรวมเข้า `main` แล้ว ตรวจว่า Production Branch ของโปรเจกต์หลักคือ main การรวมโค้ดอาจทำให้ Vercel deploy อัตโนมัติ จึงต้องเตรียมค่าก่อน
4. ใช้ source ที่ตรวจแล้วสร้าง Production deployment รอ Ready จากนั้นทดสอบสั้นด้วยบัญชีที่ได้รับอนุญาต: เข้าแอดมิน จอง อนุมัติ อีเมล LINE และเปิดหน้าย่อยตรง ๆ
5. เมื่อผลถูกต้อง ให้ LINE Login channel เป็น Published แล้วเปิด Rich Menu ของ OA จริงให้ผู้ใช้ทั่วไป ตรวจเมนูทั้งสามปุ่มจากมือถืออีกครั้ง

เกณฑ์ที่ต้องผ่านก่อนบอกผู้ใช้ว่าเปิดแล้ว

- [ ] เข้าแอดมินจริงได้ และบัญชีทั่วไปเข้าแอดมินไม่ได้
- [ ] จอง/อนุมัติ/ปฏิเสธ/รับคืน/รูป/นำเข้า/QR ทำงานในระบบทดสอบ
- [ ] เห็นเมลและ LINE ที่มือถือผู้รับจริง และตรวจรอบเตือนคืนแล้ว
- [ ] ปฏิทินสาธารณะไม่แสดงข้อมูลผู้ยืม

> หากมีปัญหา: หยุดเผยแพร่ Rich Menu ใหม่ชั่วคราว ให้ผู้ดูแลใช้ Vercel ย้อนกลับ deployment ที่เคยใช้ได้ การย้อนเว็บไม่ย้อนข้อมูลหรือ Firebase rules ต้องกู้ตามแผนแยก และแก้ค่าตั้งค่าที่ผิดก่อนเปิดต่อ

ถ้าเว็บหรือ Firebase ใช้ไม่ได้

สิ่งที่พบ	ตรวจตามลำดับ
No entrypoint found ใน smo-eq	ตรวจ Root Directory เป็นราก repo และใช้ vercel.json ล่าสุด ต้องมี app + api ไม่ใช้ smo-eq เป็น service
Build ขึ้น MISSING/INVALID	ดูชื่อตัวแปรที่แจ้ง ตรวจสะกด, Environment และ project ให้ถูก แล้ว Redeploy ไม่ปิด release check
แก้ค่าแล้วเว็บยังเหมือนเดิม	ค่าใหม่ยังไม่ใช้กับ deployment เก่า ตรวจว่าทำ Redeploy รุ่นของ environment ที่ถูกต้องแล้ว
ยังเห็นโหมดตัวอย่าง	กำลังใช้ Preview ซึ่งปิด API จริง ใช้โปรเจกต์ทดสอบแยกแบบ Production ไม่แก้ guard ของ Preview หลัก
แอดมินบอก “รหัสผ่านไม่ถูกต้อง”	ตรวจอีเมลและรหัส Firebase ของระบบนี้ ไม่ใช้ App Password ตรวจว่าหน้าเว็บชี้ Firebase project ที่ถูก และให้ผู้ดูแลช่วยรีเซ็ตผ่านช่องทางปลอดภัย
เข้าได้แต่บอกไม่มีสิทธิ์	ให้ผู้ดูแลตรวจ email verified และ admin=true ของ UID ใน project ที่ถูก แล้วออก/เข้าใหม่ การเพิ่มอีเมลผู้รับไม่ได้ให้สิทธิ์แอดมิน
ขึ้น permission-denied / รูปไม่ขึ้น	ให้ผู้ดูแลตรวจ rules, indexes, service account และ bucket รวมถึงสถานะแผน Storage อย่าเปิดฐานข้อมูลสาธารณะเพื่อแก้
หน้าเว็บเปิดได้แต่ API ไม่ทำงาน	ตรวจ routing /api/* ไป api, credentials ของ API และ Logs ขอความช่วยเหลือพร้อมเวลา/ข้อความ error โดยปิดข้อมูลลับ

> หากหน้า LIFF เจอ “Log in to Vercel” นั่นเป็นการป้องกัน deployment ไม่ใช่รหัส LINE ให้ผู้ดูแลตั้งการเข้าถึงเว็บทดสอบให้เหมาะสม ห้ามแจก password หรือ bypass token ใน Rich Menu

ถ้าอีเมลหรือ LINE ไม่มา

สิ่งที่พบ	วิธีตรวจ
App Password ไม่มีให้สร้าง	ตรวจ 2-Step Verification และนโยบายบัญชีองค์กร/Advanced Protection ให้เจ้าของบัญชีหรือ Workspace admin ช่วย
Gmail ส่งไม่ได้ / Authentication failed	ตรวจ SMTP_USER ว่าเป็นผู้ส่งตัวเดิม และ SMTP_PASS เป็น App Password 16 ตัวไม่มีช่องว่าง ถ้าเปลี่ยนรหัส Google ให้สร้างใหม่แล้ว Redeploy
ตั้ง Gmail แต่ขึ้น Apps Script	ล้าง URL ของ Google Apps Script ในหน้าแอดมิน แล้วบันทึกการตั้งค่า
บริการอีเมลตอบรับ แต่ไม่เห็นเมล	ตรวจผู้รับ Inbox/Spam/All Mail และตัวกรอง ให้เปิดดูเมลจริงก่อนนับว่าทดสอบผ่าน
LINE เข้าได้เฉพาะคนสร้าง	ตรวจ Developing/Published และ Roles ของบัญชีทดสอบ ไม่เผยแพร่ก่อนทดสอบครบ
เชื่อม OA ไม่ได้ / หา OA ไม่เจอ	ตรวจ Provider เดียวกัน สิทธิ์ admin ทั้งสอง channel และ Linked LINE Official Account
เปิด Rich Menu แล้วไม่ผูก LINE	ปุ่มต้องเป็น LIFF URL ไม่ใช่ URL Vercel ตรง ตรวจ LIFF ID, openid/profile และทำขั้นตอน consent ให้สำเร็จ
อนุมัติแล้ว LINE ไม่มา	ตรวจว่าคำขอนั้นผูก LINE จริง เพิ่มเพื่อน/ไม่บล็อก OA, token ถูก channel, LINE_ENABLED=true และโควตา OA ยังมี
เตือนไม่มาตรง 09:00	บน Hobby รอจนพ้น 09:59 แล้วให้ผู้ดูแลตรวจ Cron/Logs/คิวงานมีเฉพาะ Production และไม่เตือนรายการคืนแล้ว

> แจ้งปัญหาโดยส่งชื่อโปรเจกต์ URL วันเวลาและข้อความผิดพลาด ปิดข้อมูลผู้ยืมในภาพหน้าจอ ไม่ส่งรหัสผ่าน, App Password, token, JSON key, CRON_SECRET, OTP หรือลิงก์ยืนยันบัญชี

ภาคผนวก: ส่งให้ผู้ดูแลเทคนิค

หน้านี้ไม่ใช่ขั้นตอนที่เจ้าของเว็บต้องลองรันเอง ผู้ดำเนินการต้องมี repo, Node 24 และ credential ที่องค์กรอนุมัติ ตรวจ project เป้าหมายและสำรองก่อนทำ

1) ยืนยันอีเมล แล้วตั้ง admin claim

สร้างลิงก์ยืนยันด้วย Firebase Admin SDK ตามกระบวนการองค์กร ส่งให้เจ้าของบัญชีโดยส่วนตัว สคริปต์ด้านล่างไม่ทำ email verification และต้องใช้ credential ฝั่ง operator ที่มีสิทธิ์จัดการ Auth ซึ่งแยกจาก runtime แบบ Viewer

```
node scripts/admin-claim.mjs --project YOUR_PROJECT_ID --uid VERIFIED_ADMIN_UID
# ตรวจ dry run ให้ถูกคน/โครงการ ก่อนใช้คำสั่งจริง
node scripts/admin-claim.mjs --project YOUR_PROJECT_ID --uid VERIFIED_ADMIN_UID --apply
```

2) Deploy rules, indexes และ Storage

```
npx --yes firebase-tools@15.34.1 deploy --only
firestore:rules,firestore:indexes,storage --project YOUR_PROJECT_ID
```

รอ index พร้อม ตรวจ Firestore/Storage policies ที่ deploy จริงและ IAM ขอบเขต bucket ห้ามใช้ผล emulator อ้างแทนค่าที่อยู่บนคลาวด์

3) ข้อมูลเดิมและการเก็บเอกสารโควตา

```
node scripts/project-public-calendar.mjs --project YOUR_PROJECT_ID
# ตรวจผล dry run / สำรอง ก่อนเพิ่ม --apply เมื่อจำเป็น
```

ตรวจ publicCalendar ว่ามีเพียงข้อมูล equipment/date/status ที่อนุญาต ไม่มีข้อมูลผู้ยืม ตั้ง Firestore TTL ของ collection group `submissionLimits` บน field `expiresAt` เพื่อล้างเอกสารเก่า

4) ตรวจค่าก่อนปล่อย

```
npm run release:check -- --frontend
npm run release:check
```

ให้ credentials ผ่านระบบจัดเก็บความลับที่อนุมัติ ตัวตรวจดูรูปแบบ/ความครบถ้วน ไม่ยืนยันสิทธิ์หรือการส่งถึงจริง ทดสอบบนระบบแยก รวมคืนของ/นำเข้า/QR และตรวจ commit/CI ก่อนปล่อย ห้ามพิมพ์ secrets ลง log

ใบบันทึกค่าที่ไม่ใช่รหัสลับ

ใช้หน้านี้จดไว้ตรวจเทียบสองระบบ หรือพิมพ์ออกมาให้ทีมกรอก ไม่ต้องบันทึกรหัสผ่านหรือ token ลงหน้านี้

รายการ	ระบบทดสอบ	ระบบจริง
ชื่อ Vercel project	_____	_____
โดเมนเว็บ (https://...)	_____	_____
Production Branch	_____	_____
Firebase Project ID	_____	_____
Storage bucket	_____	_____
อีเมลบัญชีผู้ดูแลเว็บ	_____	_____
Gmail ผู้ส่ง	_____	_____
อีเมลผู้ดูแลผู้รับ	_____	_____
LINE Provider	_____	_____
LINE Login Channel ID	_____	_____
LIFF ID	_____	_____
ผู้รับผิดชอบ / วันที่ตรวจ	_____	_____

> รหัสลับ 4 ชนิดเก็บแยกในที่ปลอดภัย: Firebase service account JSON, Gmail App Password, LINE Messaging access token และ CRON_SECRET คู่มือนี้ไม่มีรหัสจริงของคุณ

หลังเปิดใช้งาน ใครตรวจอะไร

มอบหมายคนตรวจ Inbox/คิวแจ้งเตือนและ Vercel Logs เป็นประจำ ตรวจโควตา LINE/Gmail และค่าใช้จ่าย Firebase เมื่อมีการเปลี่ยนผู้ดูแลให้ถอนสิทธิ์คนเดิมและเปลี่ยนรหัสลับตามนโยบายทีม

ลิงก์ที่ใช้และคำศัพท์สั้น ๆ

ลิงก์ทุกหัวข้อกดเปิดได้จาก PDF เอกสารผู้ให้บริการอาจเปลี่ยนหลังวันที่จัดทำ หากหน้าจอไม่ตรงให้ใช้ชื่อหัวข้อค้นในเว็บทางการ

สิ่งที่ต้องเปิด	ลิงก์
Vercel Dashboard	vercel.com/dashboard
Firebase Console	console.firebase.google.com
Google Cloud Console	console.cloud.google.com
Google Security / App Password	ความปลอดภัยบัญชี Google / App Passwords
LINE OA / Developers	OA Manager / Developers Console

เอกสารทางการที่ใช้ตรวจขั้นตอน

Vercel Services: หลาย services ในโปรเจกต์เดียว

Vercel: Environment Variables และ Config / Secret

Vercel: ข้อจำกัดเวลา Cron ของแต่ละแพ็กเกจ

Firebase: เงื่อนไข Cloud Storage และแผน Blaze

Google: วิธีใช้และข้อจำกัด App Password

LINE: เปิด Messaging API ให้ OA

LINE: ผูก OA กับ LINE Login ภายใต้ Provider เดียวกัน

LINE: สร้าง LIFF และเลือก scopes

LINE: การใช้ Rich Menu

คำศัพท์: Deploy = นำเว็บรุ่นหนึ่งขึ้นให้เปิดใช้ • Redeploy = สร้างและนำขึ้นใหม่ • Domain = ชื่อที่อยู่เว็บ • Environment = ชุดแยกทดสอบ/จริง • Token/Key = รหัสให้ระบบติดต่อกัน • UID = รหัสบัญชีผู้ใช้ • CI = ชุดตรวจโค้ดอัตโนมัติ

อ้างอิงโค้ด smooa-equipment commit 50c707a และเอกสารตั้งค่าใน repo ตรวจลิงก์ทางการวันที่ 27 ต.ค. 2569 คู่มือนี้ไม่มีการเปลี่ยนค่าบนคลาวด์หรือยืนยันการส่งจริงแทนเจ้าของบัญชี